

A Project Report

On

IMPLEMENTATION OF NEURAL NETWORK MESH GENERATION ALGORITHMS FOR 3D DOMAIN

BY

ARYAN AGARWAL

2024A7PS0119H

Under the supervision of

PROF. TATHAGATA RAY

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF

CS F266: STUDY PROJECT



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI (RAJASTHAN)

HYDERABAD CAMPUS

(MARCH 2026)

ACKNOWLEDGMENT

At the very onset of this report, I would like to take the opportunity to profoundly thank Prof. Tathagata Ray for guiding me through the course of this project. Without his unwavering support, expert insights and constant feedback, this project would not have been possible.

I would also like to express my sincere gratitude to Ms. Pothuri Sowmya Sudha ma'am for her invaluable guidance and encouragement, which helped me confidently move forward through the various stages of this project.



**Birla Institute of Technology and Science-Pilani,
Hyderabad Campus**

Certificate

This is to certify that the project report entitled “**IMPLEMENTATION OF NEURAL NETWORK MESH GENERATION ALGORITHMS FOR 3D DOMAIN**” submitted by Mr. Aryan Agarwal (ID No. 2024A7PS0119H) in partial fulfillment of the requirements of the course CS F266, Study Project Course, embodies the work done by him under my supervision and guidance.

Date: March, 2026

(PROF. TATHAGATA RAY)

BITS- Pilani, Hyderabad Campus

Abstract

This project aims to design an algorithm to generate an unstructured mesh on an input contour shape. The algorithm aims to use a neural network based approach for this task, with multiple models being used in a pipeline to accomplish the successive stages of predicting the number of vertices required, the location of these vertices and the connectivity matrix of these vertices.

The first phase of the project involved exploring the problem at hand and understanding the need for an automated algorithm for creating meshes. This was accomplished by reading through the vast literature that has been written on this subject, and exploring the reasoning behind why traditional methods of generating meshes manually is not feasible in modern-day applications.

The next phase of the project involved performing research on existing mesh generation models, and understanding the progress that has already been made by the community in this domain.

This report highlights the insights gained from the aforementioned research, and summarizes the problem at hand, and the proposed solutions to it. Furthermore, it also presents the next phases of the project and how to move forward in the future.

Contents

Title page.....	1
Acknowledgements.....	2
Certificate.....	3
Abstract.....	4
1. Chapter 1	
1.1. Introduction to Meshes.....	6
1.2. Requirements for a High Quality Mesh.....	7
1.3. Triangulation Methods.....	8
2. Chapter 2	
2.1. Traditional Mesh Generation Algorithms.....	11
3. Chapter 3	
3.1. Mesh Generation using Neural Networks.....	14
3.2. Analysis of Papagiannopoulos et al. Paper.....	15
3.3. Analysis of Soman et al. Paper.....	18
Conclusion and Future Steps.....	20
References.....	21

Chapter 1

Introduction to Meshes

We start by defining what a mesh actually is. In simple terms, a mesh can be defined as a collection of vertices, edges and faces that divide a given polygon (in the 2-D domain) or polyhedron (in the 3-D domain) into smaller finite elements. These elements can be simplicial i.e. triangles or tetrahedra, or may consist of more complex shapes such as quadrilateral, hexahedra, octahedra etc. Depending on the shape of the internal elements, meshes can typically be divided into two types:

- **Unstructured Meshes:** Built by arbitrary placement and connection of vertices, and usually contain simplicial elements such as triangles (in 2-D) or tetrahedra (in 3-D).
- **Structured Meshes:** Characterised by regular, repeating geometry with numbered vertices to determine connectivity; may have more complex elements such as quadrilaterals (in 2-D) or hexahedra (in 3-D).

Both the above types have their respective pros and cons, and their unique applications. While unstructured meshes are better at meshing complex shapes and contours; they usually take more time to generate due to the lack of a repeating geometry, and might not converge. On the other hand, structured meshes are much faster to generate, and offer more control over alignment and connectivity of the elements; however, they are harder to generalized to complex shapes and are harder to locally refine once they have been generated.

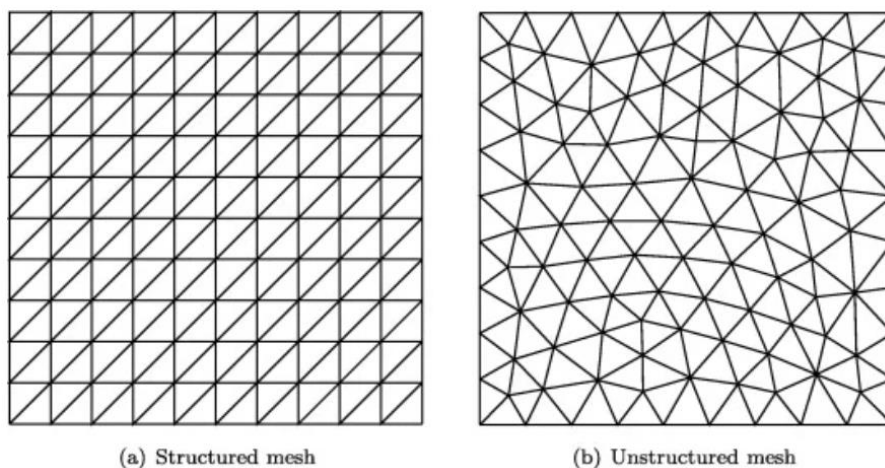


Fig 1.1 : Examples of Structured and Unstructured meshes

Meshes find applications in various areas such as in finite element analysis in architecture and construction, computational fluid dynamics, computer graphics, etc. All of these areas require the creation of meshes to cover the interior of a given 2D or 3D domain.

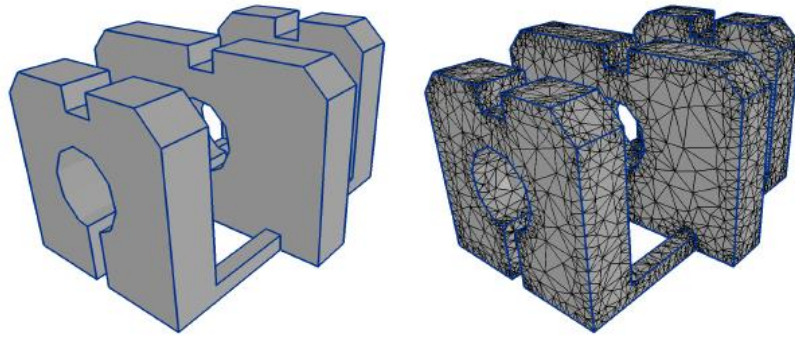


Fig 1.2 : Example of meshing a 3-D shape

Traditionally, mesh generation represented a rigorous mathematical task, and was accomplished by approximating solutions to a series of partial differential equations on the grid, most commonly Poisson's equations.

These methods, though reliable in creating meshes of decent quality, is computationally very expensive, and not feasible as the application scales, or for complex contours. Thus, there arose a need for an automated process to construct such meshes using some other method.

Requirements for a High Quality Mesh

Before discussing algorithms, we must look at the requirements that a good mesh generation algorithm must satisfy, and any constraints that they must adhere to, with respect to the entire domain as well as with respect to each element.

Contour Constraints

Input contours can become increasingly complex and impose multiple constraint on mesh generation algorithms. The obvious ones, of course, are that the algorithm should mesh the entire interior part of the domain, with all vertices and elements being entirely within the domain boundary. Also, no two edges or elements may overlap, even in part.

More complex domain constraints may arise when domains have internal edges that cannot be changed and must be included in the mesh, or when the interior has a hole that the algorithm must mesh around. Elements must not cross these boundaries.

Furthermore, elements of a mesh must conform i.e. intersecting elements must intersect only at vertices or along the entire edge/face. Two elements may not share parts of the same edge/face of a third element.

Element Quality

Elements may be constrained with regards to their shape, size, and how they vary across the domain. These constraints may change with respect to the application for which the mesh is being created and different algorithms exist that enforce one or more of these constraints better than others and are thus well-suited for a specific application.

The first constraint of a good quality element is that there must be no large angles (near 180°) or small angles (near 0°), including plane and solid angles. While large angles cause interpolated gradients to grow arbitrarily large while small angles cause approximation errors in internal calculations of the mesh connectivity, and may get rounded down to zero.

For simplicial elements, very small angles or large angles might lead to skinny elements which weaken the mesh, and violate properties of certain meshing algorithms which are built on Delaunay triangulation methods (see below).

The next constraint is on element size, and states that a good mesh generator algorithm must be able to move from very small (*fine*) elements to very large (*coarse*) elements over a relatively short distance if the need be, and should not require all elements of the mesh to be of similar size. Such meshes with large variations in element sizes are called graded meshes and are required in applications where different regions of the domain are better served by different element sizes.

Ideally, mesh generation algorithms should strive to initially create as coarse a mesh as possible since it is easier to refine a coarse mesh into a finer one by adding more vertices, as compared to vice versa.

Triangulation Methods

The earliest algorithms designed for generating meshing mostly involved finding or improving methods to efficiently triangulate a given domain. A triangulation of a finite set of points can be defined as follows:

Triangulation: A triangulation T of a finite set of points V is a simplicial complex such that V is the set of vertices in T and the union of all the simplices in T is the convex hull of V .

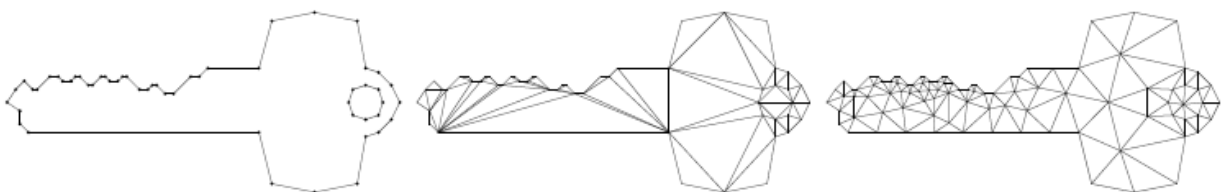


Fig 1.3 : Examples of triangulations of a key-shaped contour

The simplest form of this involves only using triangles whose vertices coincide with the vertices of the input contour; in other words, no extra internal vertices are used. An example can be seen in the first triangulation attempt on the key shape in Fig 1.3.

This method is almost always insufficient as most shapes have large cavities that simply cannot be meshed properly using such triangles, and the resulting mesh is usually badly shaped.

Steiner Triangulation

For a mesh to be made up of high-quality triangles, most algorithms use the more common Steiner triangulation. In this, additional vertices called Steiner points can be added in the cavity region which allows the creation of well-shaped triangles and a more even breakup of the domain.

An example of this type of triangulation can be seen in the second triangulation attempt of the key shape in Fig 1.3.

However, this method still does not put any constraint on the shape or size of the triangles formed, and consequently, may result in very skinny or oversized triangles being present in the final mesh, which is not ideal for many applications.

Delaunay Triangulation

The Delaunay triangulation is the gold standard for triangulation methods and is frequently used to design provably good mesh generation algorithms, because it guarantees a well-formed mesh by definition itself.

The main way that this method improves on the previous one is by enforcing a constraint on new vertices added by the algorithm, which states that no vertex of the triangulation may lie in the interior of an existing triangle's circumcircle. In other words, this rule makes sure that any new vertices added are far enough away from existing ones, so as to prevent unnecessary skinny triangles being added.

In two dimensions, it has been proven that among all possible triangulations for an input point set, Delaunay triangulation methods maximise the minimum angle in the entire set of triangles, along with optimising other element quality criteria.

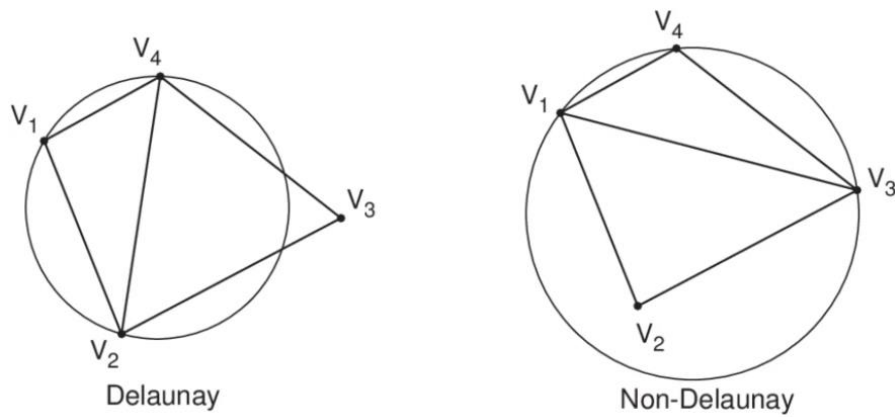


Fig 1.4 : Example of Delaunay v/s Non-Delaunay triangulations

Most early examples of mesh generation algorithms such as the advancing front method, used a Delaunay mesh generator to sequentially add newer vertices.

Constrained Delaunay Triangulation

This is a slightly restrained version of the Delaunay triangulation that is used when predefined internal edges or “hard lines” are present in the interior of the domain. Where a regular Delaunay triangulation might attempt to flip the edge if it creates Delaunay triangles, the constrained version forces the algorithm to respect these inner edges, while still maintaining the Delaunay property as much as possible.

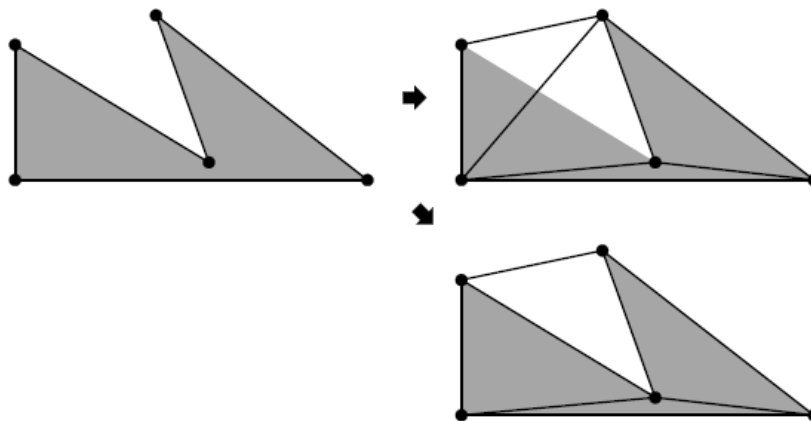


Fig 1.5 : Example of Regular v/s Constrained Delaunay triangulation

Chapter 2

Traditional Mesh Generation Algorithms

Advancing Front Method

This algorithm worked by sequentially meshing the domain, starting from the domain boundary, and then advancing inward until it meets an edge, or the opposite front. The front refers to the boundary where the meshed part of the domain meets the unmeshed part.

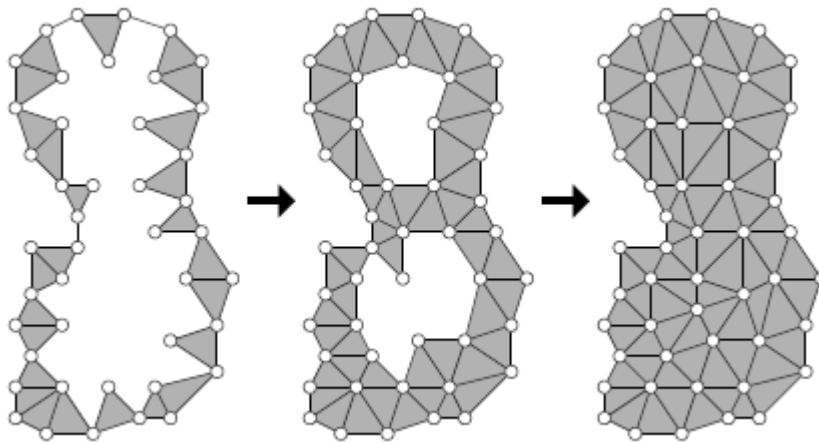


Fig 2.1 : Advancing Front Method

This method generally yields high-quality triangles, especially near the domain boundary, but may have bad elements in the places where the front collides with itself in the interior of the mesh.

This method created all the vertices and triangulated them in separate stages. However, in more advanced methods, vertex creation is now interleaved with triangulation. An example of this is Delaunay refinement algorithm, which is discussed later in this section.

Quadtree/Octree Mesh Generators

These methods overlay the domain with a background grid with a small enough resolution such that one cell of the grid covers a part of the domain that can easily be separately triangulated. To create a high-quality graded mesh, this resolution can vary across the domain, depending on the size of the domain falling into each cell.

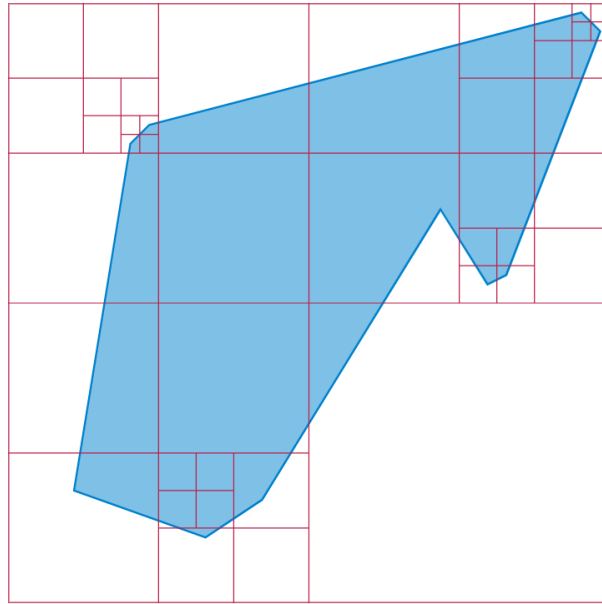


Fig 2.2 : Example of quadtree grid with atmost one contour vertex per grid cell

This method tends to form high quality triangles in the interior of the domain and worse elements near the domain boundary.

Delaunay Refinement Algorithms

While earlier Delaunay triangulation algorithms generally triangulated after adding all the vertices, which led to some bad-quality elements; Delaunay refinement algorithms work by maintaining a Delaunay triangulation throughout, and using the triangulation itself to find best locations for adding new vertices.

These algorithms maintain good quality triangles by introducing a bound on any new triangles to have a circumradius-to-shortest edge ratio of atleast B , where B is a number usually between 1 and $\sqrt{3}$.

Examples of these include L. Paul Chew's first and second Delaunay refinement algorithms and Jim Ruppert's Delaunay refinement algorithm.

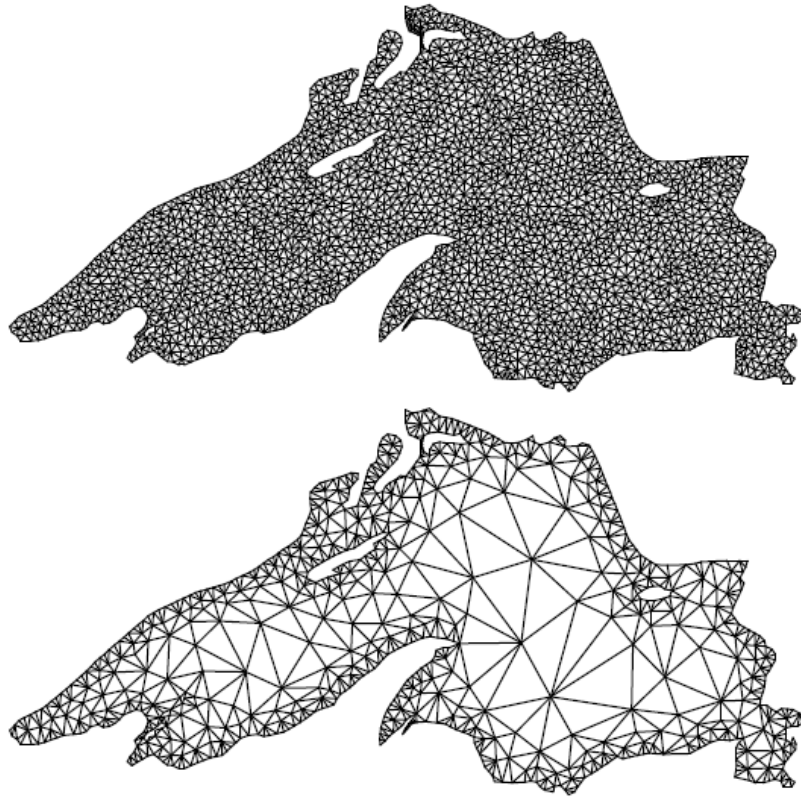


Fig 2.3 : Meshes generated by Chew's first algorithm and Ruppert's algorithm respectively

Chapter 3

Mesh Generation using Neural Networks

Moving from solving PDEs to using neural networks represents a massive change in how the mesh generation problem is approached, and drastically reduces the time and computational power required to mesh a given shape. While traditional methods solve PDEs, and iteratively generate the mesh, one vertex after another; neural networks, once they have been trained on sufficient data, can instantly generate meshes of comparable quality to the best Delaunay mesh generators, using internal parameters learned from the training data.

Mesh generation using neural networks have several advantages over the traditional mathematical methods:

- **Speed:** Since neural network based mesh generation does not actually require approximation of solutions to PDEs, or other complicated maths, it is much faster than all the other mesh generation algorithms discussed above.
- **Efficiency:** Neural networks are much better at refining and altering a mesh, once generated, and does not need to regenerate an entire mesh for small alterations to the input contour, making it highly useful in applications where the object to be meshed undergoes slight changes in shape and size often.
- **Versatility:** Traditional mesh generation struggles when the domain is complex or has imperfections. However, neural network based generators can effectively learn to handle such issues and fix the geometry before meshing.

However, there are also some issues that neural network mesh generation methods entail. These can be enumerated as follows:

- **'Black Box' Risk:** Unlike traditional Delaunay methods, neural networks do not guarantee mathematical accuracy and might misplace a vertex, leading to overlapping triangles or tangled meshes that will reduce the mesh quality.
- **Data Dependency and Training Cost:** Neural networks require a massive amount of data to train, and this might not be feasible for many applications where such amounts of training data is not available. Furthermore, training the model on large datasets also represents a significant computational cost.
- **Generalization:** While traditional solvers can usually adapt to any contour or shape by adjusting the PDEs accordingly, neural network models that have been primarily trained on a certain class of contours might not be able to generalize to unseen shapes or domains without additional training.

These problems, and others, represent the subject of ongoing research in this field, and newer algorithms are continuously being developed to tackle one or more of these problems. Despite the setbacks and obstacles, neural networks are the future of mesh generation, and will soon replace the traditional methods in real world applications.

To explore and gain a deeper understanding of the reasoning and methods used by such algorithms, I studied and analysed recent research papers pertaining to this topic. The rest of this section will discuss the crux of two such papers.

Analysis of Paper I by Papagiannopoulos et al.

The first paper I read was a paper titled ‘How to teach neural networks to mesh: Application on 2-D simplicial contours’ by Alexis Papagiannopoulos, Pascal Clausen and François Avellan, published in *Neural Networks* (2021).

This paper approached the mesh generation problem by decomposing it into multiple steps, and tackling each stage sequentially. The pipeline was as follows:

- **Preprocessing:** Consists of feature transformation of the input contour to scale and rotate it using Procrustes superimposition, with respect to a unit polygon.
- **NN1:** First neural network; predicts the number of internal vertices (Steiner points) based on input target edge length.
- **NN2:** Second neural network; predicts the spatial coordinates of each vertex using a grid-based scoring system.
- **NN3:** Third neural network; the actual mesh generation model that outputs a connectivity matrix with probabilities of connecting specific vertices, which is then used by a special triangulation algorithm to form the final mesh.

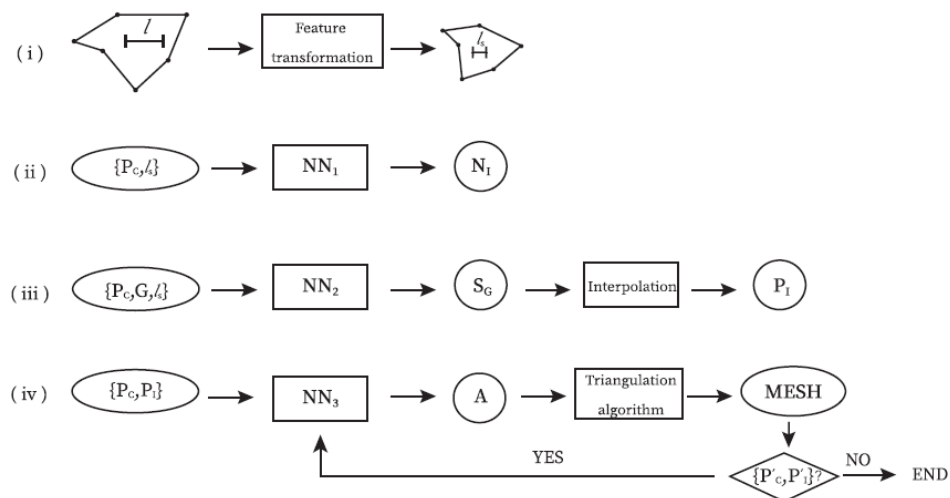


Fig 3.1 : Complete pipeline of the Paper I mesh generation algorithm

NN1 – Predicts Interior Vertex Count

This is the first Neural Network, and it's responsible for determining the number of interior vertices that must be inserted within the domain. It takes input geometric features along with the desired edge length of the mesh. With these inputs, NN1 estimates the number of mesh vertices required to achieve a mesh with suitable density, as a mesh with low density makes the mesh coarse, while high-density meshes result in unnecessary computational cost.

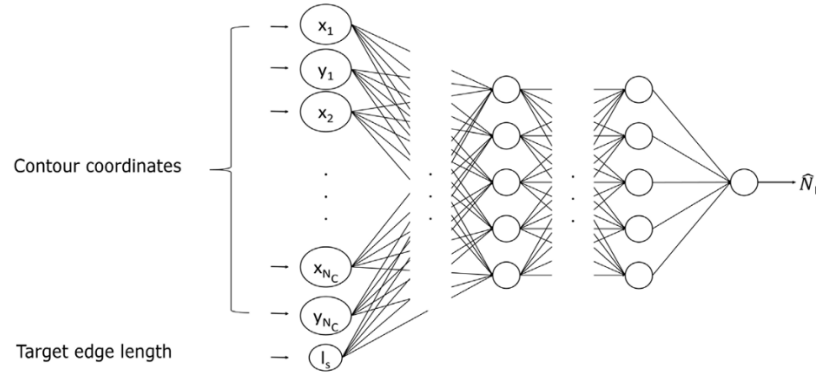


Fig 3.2 : Diagrammatic representation of the working of NN1

NN2 - Predicts Interior Vertex Positions

The main purpose of NN2 is to determine the spatial positions of these vertices. A grid is generated over the convex hull, and each grid point is evaluated using a scoring mechanism. The score is assigned based on its suitability for each vertex. NN2 then selects the vertices with the minimum scores as the positions for the interior vertices. This process ensures that the vertices are distributed uniformly throughout the domain while respecting the geometrical boundaries.

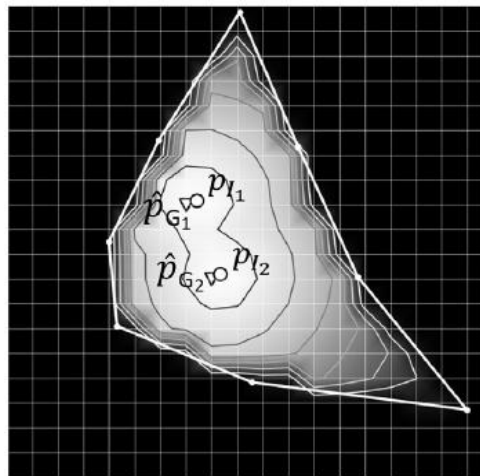


Fig 3.3 : Working of NN2; Points p_{l1} and p_{l2} have minimum score among all possible points

NN3 - Predicts Connectivity between Nodes

Finally, the third neural network, NN3, determines the connectivity relationships between vertices to construct the mesh elements. Using the contour vertices and the predicted interior vertices, NN3 predicts how these vertices should be connected to form triangular elements. The network outputs a connection table that defines the relationships between vertices, which is then used to construct the final triangular mesh. In this final step, the algorithm ensures that the generated mesh maintains a valid topology and satisfies the structural requirements of a high-quality mesh, and each grid point is evaluated using a scoring mechanism.

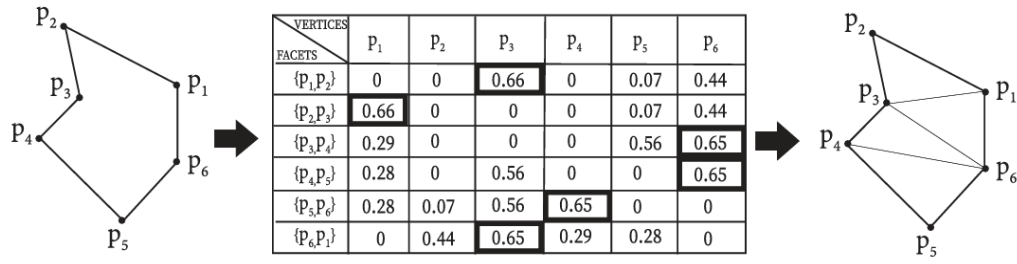


Fig 3.4 : Connection table generated by NN3

To evaluate the performance of this mesh-generating model, certain error metrics have been calculated to compare the quality of the generated and predicted mesh. These metrics indicate how closely the neural-network-generated mesh matches the quality of meshes produced by traditional meshing algorithms.

- **Mean Element Quality**

$$q_{mean} = \frac{1}{N_{el}} \sum_{i=1}^{N_{el}} q_i$$

- **Worst Element Quality**

$$q_{worst} = \min(q_i)$$

- **Relative Error Metrics**

$$e_{mean} = \frac{|q_{mean}^{pred} - q_{mean}^{ref}|}{q_{mean}^{ref}}$$

$$e_{worst} = \frac{|q_{worst}^{pred} - q_{worst}^{ref}|}{q_{worst}^{ref}}$$

Analysis of Soman et al. Paper

The second paper I studied was a paper titled ‘Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries’ by Sumedh Soman and Ninad Mehendale, published in *Neural Computing and Applications* (2024).

This paper represented a significant advancement in the neural network mesh generation landscape by introducing a Conditional Generative Adversarial Network (CGAN) based mesh generator, which achieved much faster rates as compared to conventional solvers.

The inputs for this algorithm are the convex hull (geometry) of the domain and the desired mesh edge length, which are given to the CGAN architecture. This generator then learns to predict the coordinates of mesh nodes inside the domain such that their distribution maximizes the number of high-quality meshes present in the training dataset.

For training the neural network, the paper uses DistMesh, a well-known mesh generation algorithm based on force-displacement principles, with an Adam optimizer. The algorithm iteratively adjusts the positions of the nodes until the edges reach an equilibrium that produces high-quality meshes with the desired edge length. By using these meshes as the base-level training data, these meshes enable the network to map geometric features to high-quality node distributions.

Once the node positions have been finalized, a Delaunay triangulation algorithm is applied to connect the nodes and construct the final mesh elements.

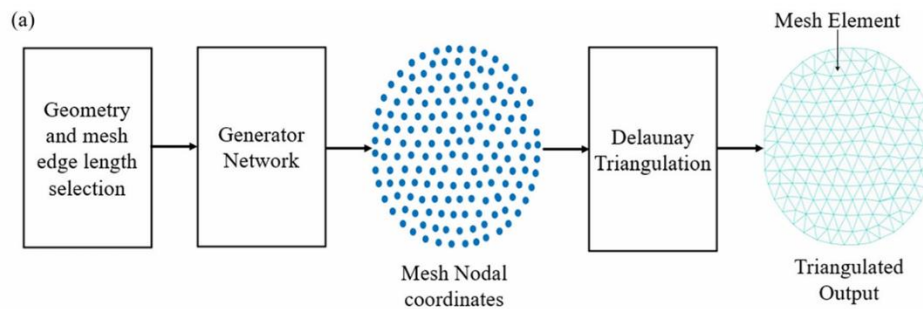


Fig 3.5 : Complete pipeline of the Paper II mesh generation algorithm

This algorithm, therefore, combines the advantages of machine learning to efficiently determine the placement of mesh nodes, and classical computational geometry ensures a valid mesh topology and prevents intersecting elements.

During training, the neural network is optimized by minimizing errors using an error loss function that measures the difference between the predicted node coordinates and the node coordinates obtained from the reference meshes generated using DistMesh.

The expression for the loss function is

$$J = \frac{-1}{N} \sum_j m_j w_j (Y_j - T_j)^2$$

Where:

- N = total number of training samples
- m_j = masking parameter used to indicate whether a node contributes to the loss
- w_j = weighting factor for the j^{th} node
- Y_j = predicted output from the neural network
- T_j = target (reference) value from the training mesh dataset

This loss function calculates the weighted squared error between the predicted and reference node positions. The weights allow certain nodes to contribute more strongly, thereby increasing the overall accuracy of the generated mesh.

Overall, the paper achieved many advancements over previous algorithms, including the following:

- 75% faster than DistMesh, and 13x times faster than ANSYS Mechanical Meshing.
- <1% difference in quality between modeled meshes and training meshes.
- Capable of meshing for both 2D and 3D domains.

Conclusion and Future Steps

In summary, this project has been an exciting deep dive into the world of mesh generation so far, and I am looking forward to continue learning more about the subject, as well as start building and experimenting with the above algorithms myself, as well as start brainstorming ideas for my own algorithm, if feasible.

The post-midsem phase of this project will focus on the actual implementation of neural network based mesh generation algorithms for the 3-D domain. Initially, the algorithms proposed in Paper II by Soman and Mehendale will be tested on basic and well-defined 3D geometries like a teacup, torus (doughnut) and other standard shapes. Testing on these shapes helps us validate the implementation as they possess simple geometric properties.

However, the implementation will follow a structure that is more similar to Paper 1 by Papagiannopoulos et al. where we utilise different neural networks for various tasks. Once the model is trained, we will use Delaunay triangulation to connect these vertices. Then these generated meshes will be evaluated on the different quality metrics as enumerated above.

While the initial experiments focus on regular and smooth geometries, a major limitation of such approaches is the difficulty in handling complex and irregular shapes. Real-world geometries often comprise of irregular and inconsistent boundaries and edges thus increasing the level of detail which aren't easily captured by the models trained on simple shapes.

To address this, the future scope of work involves modifying and developing the existing implementation methods to generalize mesh generation beyond simple domains. The end goal is to develop a meshing framework capable of generating high-quality meshes for any given 3-D shapes, irrespective of its complexity.

References

1. Shewchuk, J. R. (2012). *Lecture Notes on Delaunay Mesh Generation*. University of California, Berkeley.
2. Papagiannopoulos, A., Clausen, P. & Avellan F. (2021). How to teach neural networks to mesh: Application on 2-D simplicial contours. *Neural Networks (136)*, 152–179.
<https://doi.org/10.1016/j.neunet.2020.12.019>
3. Soman, S. & Mehendale, N. (2024). Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries. *Neural Computing and Applications (36)*, 1805-1813.
<https://doi.org/10.1007/s00521-023-09119-2>